

Open Things Access Protocol (OTAP)

HTTPS PUSHSDK Access Specifications

About this Document

- This Document includes instructions for using and managing the Product. Pictures, charts, images and all other information hereinafter are for description and explanation only. Unless otherwise agreed, our company makes no warranties, express or implied.
- Please use this Document with the guidance and assistance of professionals trained in supporting the Product.

Trademarks Acknowledgment

All trademarks and logos mentioned are the properties of their respective owners.

LEGAL DISCLAIMER

- TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, THIS DOCUMENT AND THE PRODUCT DESCRIBED, WITH ITS HARDWARE, SOFTWARE AND FIRMWARE, ARE PROVIDED "AS IS" AND "WITH ALL FAULTS AND ERRORS". OUR COMPANY MAKES NO WARRANTIES, EXPRESS OR IMPLIED, INCLUDING WITHOUT LIMITATION, MERCHANTABILITY, SATISFACTORY QUALITY, OR FITNESS FOR A PARTICULAR PURPOSE. THE USE OF THE PRODUCT BY YOU IS AT YOUR OWN RISK. IN NO EVENT WILL OUR COMPANY BE LIABLE TO YOU FOR ANY SPECIAL, CONSEQUENTIAL, INCIDENTAL, OR INDIRECT DAMAGES, INCLUDING, AMONG OTHERS, DAMAGES FOR LOSS OF BUSINESS PROFITS, BUSINESS INTERRUPTION, OR LOSS OF DATA, CORRUPTION OF SYSTEMS, OR LOSS OF DOCUMENTATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), PRODUCT LIABILITY, OR OTHERWISE, IN CONNECTION WITH THE USE OF THE PRODUCT, EVEN IF OUR COMPANY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES OR LOSS.
- YOU ACKNOWLEDGE THAT THE NATURE OF THE INTERNET PROVIDES FOR INHERENT SECURITY RISKS, AND OUR COMPANY SHALL NOT TAKE ANY RESPONSIBILITIES FOR ABNORMAL OPERATION, PRIVACY LEAKAGE OR OTHER DAMAGES RESULTING FROM CYBER-ATTACK, HACKER ATTACK, VIRUS INFECTION, OR OTHER INTERNET SECURITY RISKS; HOWEVER, OUR COMPANY WILL PROVIDE TIMELY TECHNICAL SUPPORT IF REQUIRED.
- YOU AGREE TO USE THIS PRODUCT IN COMPLIANCE WITH ALL APPLICABLE LAWS, AND YOU ARE SOLELY RESPONSIBLE FOR ENSURING THAT YOUR USE CONFORMS TO THE APPLICABLE LAW. ESPECIALLY, YOU ARE RESPONSIBLE, FOR USING THIS PRODUCT IN A MANNER THAT DOES NOT INFRINGE ON THE RIGHTS OF THIRD PARTIES, INCLUDING WITHOUT LIMITATION, RIGHTS OF PUBLICITY, INTELLECTUAL PROPERTY RIGHTS, OR DATA PROTECTION AND OTHER PRIVACY RIGHTS. YOU SHALL NOT USE THIS PRODUCT FOR ANY PROHIBITED END-USES, INCLUDING THE DEVELOPMENT OR PRODUCTION OF WEAPONS OF MASS DESTRUCTION, THE DEVELOPMENT OR PRODUCTION OF CHEMICAL OR BIOLOGICAL WEAPONS, ANY ACTIVITIES IN THE CONTEXT RELATED TO ANY NUCLEAR EXPLOSIVE OR UNSAFE NUCLEAR FUEL-CYCLE, OR IN SUPPORT OF HUMAN RIGHTS ABUSES.
- IN THE EVENT OF ANY CONFLICTS BETWEEN THIS DOCUMENT AND THE APPLICABLE LAW, THE LATTER PREVAILS.

Contents

1. Scope 1
2. Normative References 1
3. Terms, Definitions, and Abbreviations 1
3.1 Terms and Definitions 1
3.2 Abbreviations 1
3.3 Encoding 2
3.4 URL Keywords 2
4. HTTPS/HTTP Introduction 2
4.1 Request Messages 2
4.2 Response Messages 2
4.3 Data Format 3
4.4 HTTPS Interaction Process 5
5. HTTP Data Encryption 5
5.1 Encryption Method 5
5.2 Encryption Process 6
5.3 Encryption Key 6
5.4 Encryption Message 6
6. System Architecture 8
7. Function Flow 8
7.1 Platform Central Address Configuration 8
7.2 Registration Authentication 10
7.3 Command Operation 13
7.4 Event Upload 15
7.5 Device Logout 15
7.6 Real-Time Live View 17
7.7 Two-way Audio 21
7.8 Video Intercom 24
7.9 Local Upgrade 25

1. Scope

This document specifies the transmission, control, and security requirements for IoT device access, as well as technical specifications for transmission and protocol interfaces.
It applies to the design of IoT device access solutions, system testing, acceptance, and related device development.

2. Normative References

The following are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition (including any amendments) applies.
1) ISO 8601:2004 Data elements and interchange formats
2) IETF RFC8446 The Transport Layer Security (TLS) Protocol Version 1.3
3) IETF RFC2044 UTF-8, a transformation format of Unicode and ISO 10646
4) ISO/IEC 30141:2018 INTERNET OF THINGS (IoT) – REFERENCE ARCHITECTURE
5) IETF RFC2616 Hypertext Transfer Protocol -- HTTP/1.1
6) IETF RFC2617 HTTP Authentication: Basic and Digest Access Authentication
7) IETF RFC2818 HTTP Over TLS
8) IETF RFC7616 HTTP Digest Access Authentication
9) IETF RFC7233 Hypertext Transfer Protocol (HTTP/1.1): Range Requests

3. Terms, Definitions, and Abbreviations

3.1 Terms and Definitions

3.1.1 PUSH

The platform has a fixed IP and port, with the device acting as the client and the platform as the server. After configuring the platform's IP and port, the device actively connects to and accesses the platform.

3.1.2 Event

Information actively reported by the device. This type of information is reported in real-time and promptly processed by the platform. If the device’s network is interrupted, the information can be cached and then reported once the connection is restored.

3.2 Abbreviations

The following acronyms are applicable to this document:
IP: Internet Protocol
JSON: JavaScript Object Notation
UTF-8: 8-bit Unicode Transformation Format
URL: Uniform Resource Locator
opt: Optional
req: Required

3.3 Encoding

Data packets transmitted between the device and the platform are encoded using UTF-8.

3.4 URL Keywords

Keyword	Data Type	Remarks
DEVID	string	Device serial number (can consist of letters, numbers, or a combination of both).

4. HTTPS/HTTP Introduction

PUSH is based on HTTPS/HTTP protocol. This chapter provides a brief introduction to HTTPS/HTTP protocol. **You may skip this section if you are already familiar with HTTPS/HTTP protocol**.
HTTPS/HTTP typically operates on TCP/IP connections, with 443 the default HTTPS port and 80 for HTTP. However, the communication port for HTTPS/HTTP may be modified through **communication address configuration**.

4.1 Request Messages

Request messages include a request line (request method + request URL + HTTP protocol and version), request headers, and request body. The request method, request URL, and request body are defined according to the business protocol. See Figure 1 for an example:

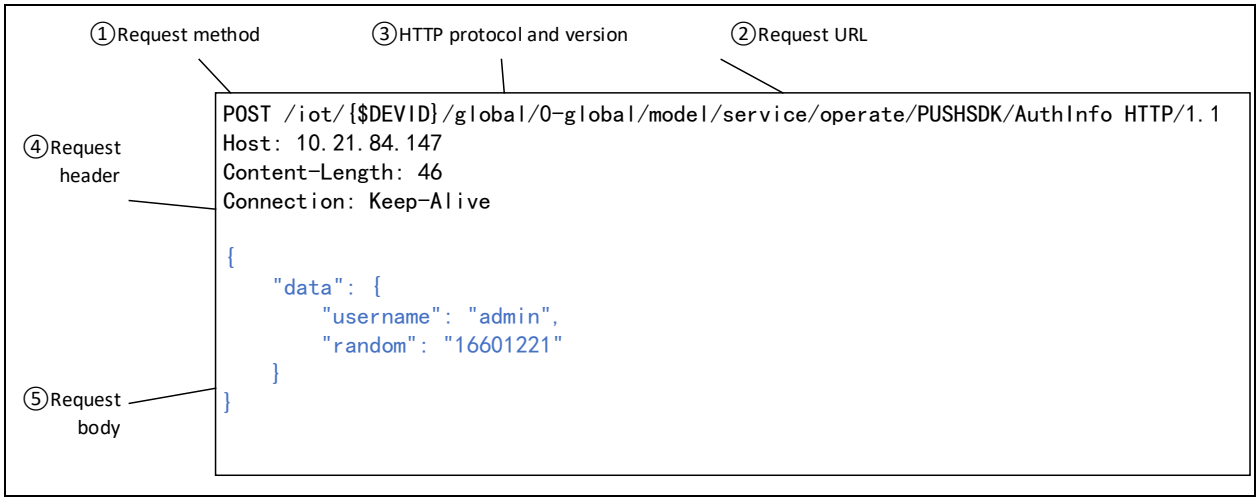


Figure 1 HTTP Request Message

4.2 Response Messages

Response messages include a response line (HTTP protocol and version + status code and status description), response headers, and response body. The response body is defined in accordance with the business protocol. See Figure 2 for an example:

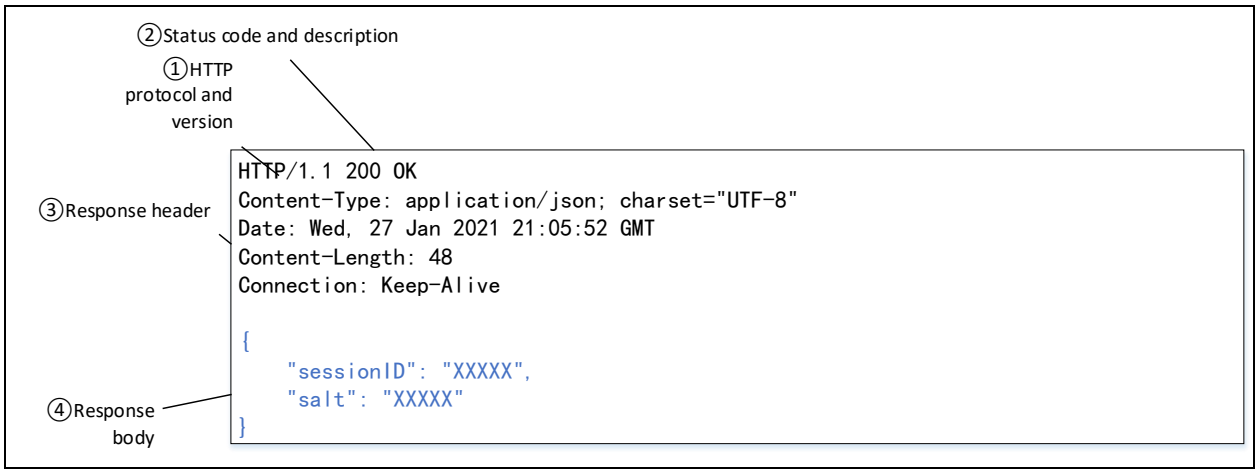


Figure 2 HTTP Response Message

4.3 Data Format

HTTPS/HTTP uses three methods for message transmission: JSON message, binary data, and MIME_boundary.

A JSON message is shown below (Content-Type: application/json):

```
HTTP/1.1 200 OK
Content-Type: application/json; charset="UTF-8"
Date: Wed, 27 Jan 2021 21:05:52 GMT
Content-Length: 48
Connection: Keep-Alive

{
    "status": 200,
    "code": "0x00000000",
    "errorMsg":"Succeeded."
}
```

Binary data is shown below (Content-Type: application/octet-stream):

```
HTTP/1.1 200 OK
Content-Type: application/octet-stream
Date: Wed, 27 Jan 2021 21:05:52 GMT
Content-Length: 4460
Connection: Keep-Alive

.....``.....C.....        .
..
..... $.' ",#..(7),01444.'9=82<.342...C.                    ....
```

MIME_boundary is shown below (1. Content-Type: multipart/form-data; 2. For details, see *IETF RFC7233 Hypertext Transfer Protocol (HTTP/1.1): Range Requests*; 3. Pay special attention to the carriage return and line feed \r\n in MIME_boundary, which must align with the standard definition.)

```
HTTP/1.1 200 OK
Content-Type: multipart/form-data; boundary=MIME_boundary      // There is a space between form-data; and boundary=; the content after boundary= can be any string but must not match the payload content.
Content-Length:4840      // Refers to the total length from the beginning of File 1 to the end of the entire
```

interaction boundary (from --MIME_boundary to --MIME_boundary--).

--MIME_boundary //Header boundary of File 1
Content-Disposition: form-data; name="ResponseStatus";
Content-Type: application/json
Content-Length:48

{
"status": 200,
"code": "0x00000000",
"errorMsg":"Succeeded."
}
--MIME_boundary //Header boundary of File 2
Content-Disposition: form-data; name="CapturePicture";
Content-Type: image/jpeg //Transmitted according to the actual binary content format. See *"IETF RFC2616 Hypertext Transfer Protocol"* for details.
Content-Length: 4460

.....``.....C..... .
..
..... $.' ",#..(7),01444.'9=82<.342...C. ....
--MIME_boundary-- // End boundary, indicating the end.

4.4 HTTPS Interaction Process

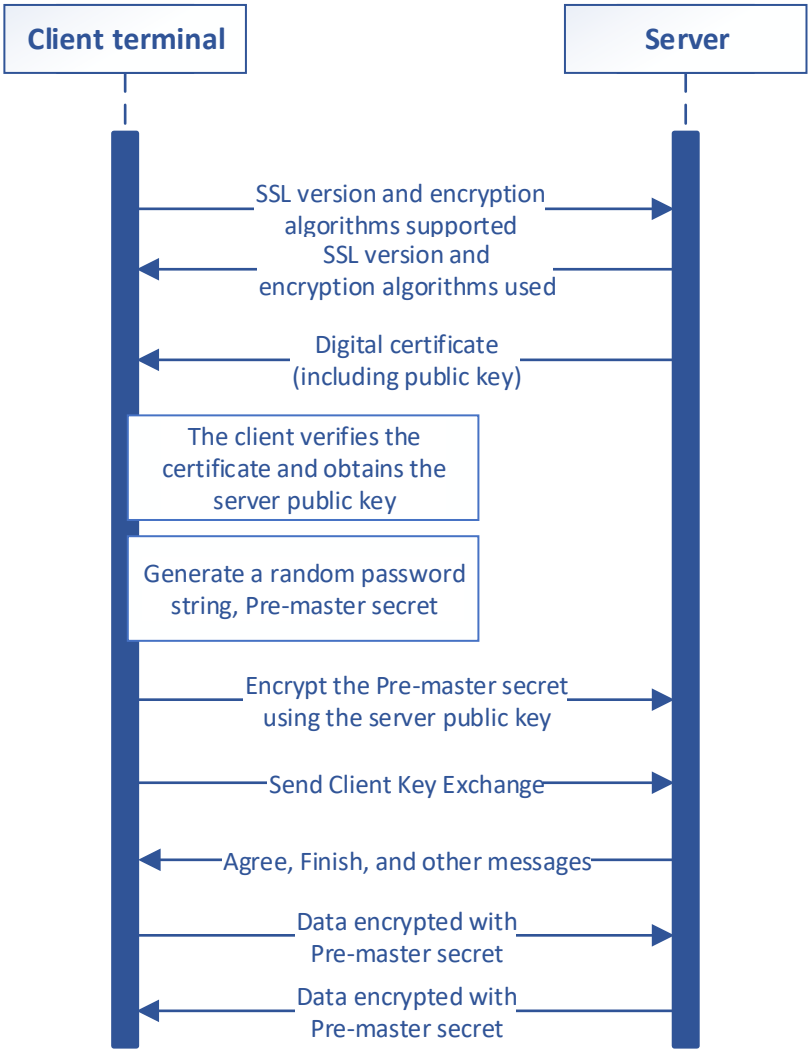


Figure 3 HTTPS Interaction Process

Details about the process governing HTTPS interaction can be found in *IETF RFC 2818 HTTP Over TLS*.

5. HTTP Data Encryption

When business functions involve sensitive fields, encryption is required. However, some IoT devices may be unable to implement HTTPS or may experience performance limitations due to hardware constraints. This section provides further explanation.

This section primarily describes encryption requirements for JSON messages. If a message in an interaction contains sensitive information fields/data, the entire data of that interaction (both request and response JSON messages) must be encrypted.

5.1 Encryption Method

1. Uniformly add a query parameter at the end of the URL to indicate the encryption scheme version of the current

protocol.

***Notes*:** The query parameters mentioned here refers to the URL parameters rather than the URL field in the request/response messages. Involved URLs are:

Get Command (POST /iot/${DEVID}/global/0-global/model/service/operate/PUSH/CommandRequest);
Command Result (POST /iot/${DEVID}/global/0-global/model/service/operate/PUSH/CommandResult);
Event Upload (POST /iot/${DEVID}/global/0-global/model/service/operate/PUSH/Event).

Query Parameter	Data Type	Value Range	Description
security	integer	[3,4]	1. When security = 3 indicates that the JSON message is fully encrypted using AES128 CBC mode. 2. When security = 4 indicates that the JSON message is fully encrypted using AES256 CBC mode. 3. If security is not specified, the data is not encrypted.
iv	string	A random string no longer than 32 characters	It represents the initialization vector and is a required parameter when security=3 or 4.
random	string	A random number for encryption with a fixed length of 16 bytes changed for each interaction.	This is a required parameter when security=3 or 4 to generate the encryption key.

2. The initialization vector for the CBC mode algorithm is generated by the client, transmitted to the device in plaintext via the URL parameter. The initialization vector is 16 bytes, with each byte represented in hexadecimal and converted to character format (length 32) during transmission. The device must convert it back to hexadecimal bytes during computation.

Notes: CBC mode does not require block-size cyclic encryption, and the initialization vector changes after each encryption. Therefore, if two or more nodes in a message need encryption, the initial vector must be copied during encoding to ensure that the same one is used.

5.2 Encryption Process

The encryption process for source data is as follows: Source data -> Base64 -> AES128 or AES256 CBC mode with PKCS7 padding.

5.3 Encryption Key

The AES algorithm key is computed using the PBKDF2 method:

1. DK = PBKDF2(Password,Salt,c,dkLen,SHA256())
   Salt is salt, *c* is iterations, and *dkLen* is a fixed length of 64.
2. Password = HexToString(SHA256(username + salt + password)) + random;
3. The final result is:
   DK = PBKDF2(HexToString(SHA256(username + salt + password)) + random, salt, iterations, 64, SHA256()
   Here, random needs to change with each interaction.

***Notes*:** When using HTTP data encryption, the username and password must be configured for the device in advance via the PUSH communication address configuration.

5.4 Encryption Message

For the server, if the URL contains the fields "security" and "iv", both request and response messages need to be encrypted (encrypting the JSON data message but not the HTTP message header).

Since the original JSON message header's Content-Type: application/json becomes binary data after encryption, making the Content-Type application/octet-stream.

6. System Architecture

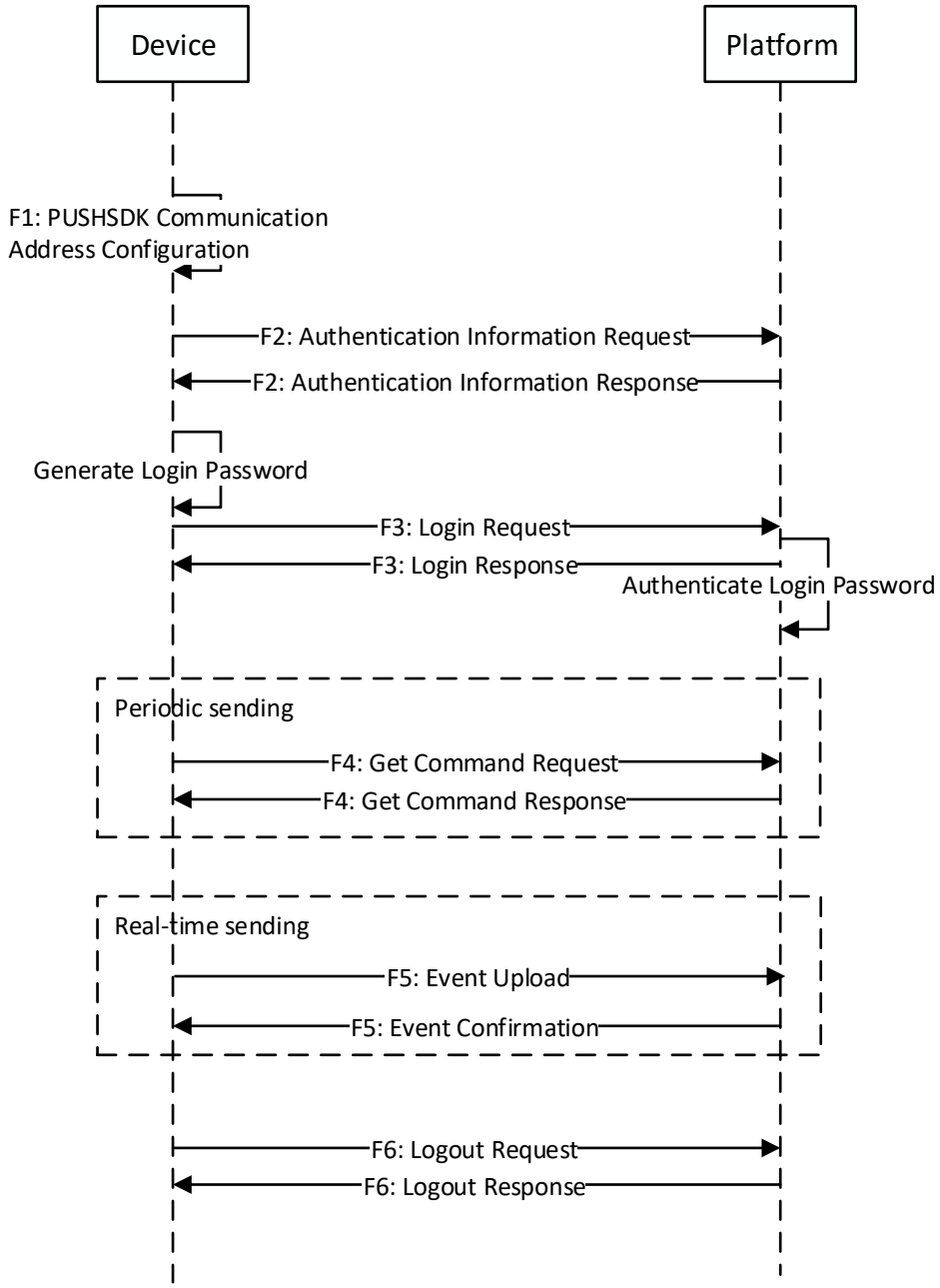


Figure 4 PUSH Interaction Process

7. Function Flow

7.1 Platform Central Address Configuration

Refer to **F1: PUSH Communication Address Configuration** in Chapter 6 System Architecture**.** Before PUSH interaction, the relevant platform information must first be shared with the device so that it can actively communicate with the platform. PUSH parameters can be configured for the device via the **ISAPI protocol**:

API Function	Get/Configure PUSH Parameters
Request URL	GET or PUT /ISAPI/System/Network/PUSHCfg?format=json

Response Message	{
	"enabled": true, /*req, boolean, whether to enable PUSH communication, desc: disabled by default*/
	"addressingFormatType": "ipaddress", /*req, string, address format, enum:[ipaddress#IP address, hostname# domain name]*/
	"hostName": "www.baidu.com", /*opt, string, domain name, range:[1,128], format:hostname, dependency:(addressingFormatType,eq,hostname)*/
	"ipAddress": "10.11.12.13", /*opt, string, IPv4 address, range:[1,16], format:ipv4, dependency:(*addressingFormatType*,eq,ipaddress)*/
	"ipv6Address": "::", /*opt, string, IPv6 address, range:[1,128], format:ipv6, dependency:(*addressingFormatType*,eq,ipaddress)*/
	"protocolType": "http", /*req, string, protocol type, enum:[http#HTTP protocol, https#HTTPS protocol]*/
	"httpPortNo": 80, /*opt, integer, HTTP registration port, range:[1,65535], dependency:(*protocolType*,eq,http, required), desc: default port 80*/
	"httpsPortNo": 443, /*opt, integer, HTTPS registration port, range:[1,65535], dependency:(*protocolType*,eq,https, required), desc: default port 443*/
	"webSocketPortNo": 80, /*opt, integer, WebSocket port, range:[1,65535], dependency:(*protocolType*,eq,http, optional), desc: default port 80, port can be configured to the same port as HTTP port*/
	"webSocketSPortNo": 443, /*opt, integer, WebSocket over TLS port, range:[1,65535], dependency:(*protocolType*,eq,https, optional), desc: default port 443, port can be configured to the same port as HTTPS port*/
	"notCertifiedEnabled": false, /*opt, boolean, whether to enable non-certified mode, desc: Default value is false. When the field is true, username and password are ineffective and may be omitted; when field “enabled” is true and this field is false or absent, username and password must be provided. This field is currently not supported by the device, meaning the device currently only supports certified mode.*/
	"username": "admin", /*opt, string, platform username, range:[1,64], desc: sensitive information encryption is required*/
	"password": "aksj13579", /*write-only, opt, string, platform password, range:[0,64], desc: sensitive information encryption is required*/
	"status": "online", /*read-only, opt, string, platform connection status, enum:[online# connected, offline# disconnected]*/
	"httpsCertificateEnabled": true /*opt, boolean, whether to enable HTTPS certificate verification*/
	"urlPreffix": "api/v1", /*opt, string, URL prefix, range:[0,64], desc: only supported by security inspection devices*/
	}

Remarks	When *notCertifiedEnabled* is true, HTTP communication will not perform registration authentication or HTTP data encryption. This means the device can directly interact with the platform for command operations, event uploads, and video intercoms without the registration authentication process.

7.2 Registration Authentication

Refer to **F2: Authentication Information Request** in Chapter 6 System Architecture. Before the device logs into the platform, it must first obtain authentication information, which can be acquired via the following API.

API Function	Obtain Authentication Information
Request URL	POST /iot/${DEVID}/global/0-global/model/service/operate/PUSH/AuthInfo

Request Messages	{
	"data":{
	"securityVersion": [3,4] /*opt, array, supported HTTP encryption algorithm versions, desc:[3,4] indicates supporting encryption algorithm versions 3 and 4*/
	}
	}

Response Messages	{
	"data":{
	"challenge": "15f1739e9a465cc048575b7d39a312fd54364399739e9a465cc048575b798753", /*req, string, challenge string, range:[64,64]*/
	"iterations": 500, /*req, integer, number of iterations, range:[500,5000]*/
	"salt": "e43edaeac631e5db1c331046294a3aee6787bea7d0a72b36ff405b83e4e6f9f3", /*req, string, salt value, range:[64,64]*/
	"isDataEncrypt": true, /*req, boolean, whether HTTP data transmission requires encryption (applicable for both HTTP and HTTPS)*/
	"securityVersion": [3,4] /*opt, array, HTTP encryption algorithm version (applicable for both HTTP and HTTPS), desc:[3,4] represents supporting for encryption algorithm versions 3 and 4*/
	}
	}

Remarks for Response	In the registration authentication process, the challenge validity timeout is 3 * *commandInterval* (command request interval), and it is only valid for a single request and cannot be reused. In the negotiation request sent by the device to the platform, if HTTP communication is used, the highest version supported by both parties, based on the *securityVersion* negotiation field, will be used for full HTTP message encryption. This means that all subsequent interaction requests must be encrypted.

Refer to **F3: Login Request** in Chapter 6 System Architecture. The device logs in to the platform through the following API.

API Function	Log In

Request URL	POST /iot/${DEVID}/global/0-global/model/service/operate/PUSH/Login

Request Messages	{
	"data":{
	"username": "admin", /*opt, string, username, range:[1,64]*/
	"loginPassword": "36fe22a754dc3983ee233b9a346dfe04b283bc472e0350c664568c5cd5bce2e215f1739e9a465cc048575b7d39a312fd54364399739e9a465cc048575b798753", /*opt, string, login password, range:[128,128], desc: The *loginPassword* calculated by PBKDF2 is a 64-character hexadecimal data. When converted into an ASCII string, its length becomes 128 characters.*/
	}
	}

Remarks for Request	The *loginPassword* calculation method uses PBKDF2: 1. DK = PBKDF2(Password, Salt, c, dkLen, SHA256()) *Salt* is salt, *c* is iterations, and *dkLen* is a fixed length of 64. 2. Password = HexToString(SHA256(username + salt + password)) + challenge 3. The final result: DK = PBKDF2(HexToString(SHA256(username + salt + password)) + challenge, salt, iterations, 64, SHA256()) ***Notes:*** 1. When using *loginPassword* for authentication, the username and password must be configured for the device in advance via the PUSH communication address. 2. The following device models and firmware versions use the SHA1() parameter in the PBKDF2 function instead of SHA256(): ● DS-K1T6240MGB-QR (V1.6.1_build241226) ● DS-K1T502DBWX-CQR (V1.9.1_build250317) ● DS-K1T321MFWX (V3.9.2_build250123) ● DS-K1T321MFWX (V3.9.20_build250604) ● DS-K1T341BM (V3.2.32_build250221) ● DS-K1T341BMW (V3.2.32_build250221) ● DS-K1T342MFWX-E1 (V3.16.1_build250320) ● DS-K1T341CMFW (V3.3.15_250410) ● DS-K1T341CMFW (V3.3.40_251029) ● DS-K1T341CMF (V3.3.8_240617) ● DS-K1T673DX (V4.25.0_build240807) ● DS-K1T343MFWX-B (V3.21.0_240723) ● DS-K1T341CMF (V3.3.8_231107) ● DS-K1T341CMFW (V3.3.8_231107)

Response Success Message	{
	"status": 200, /*req, integer, status code*/
	"code": "0x00000000", /*req, string, error code*/
	"errorMsg": "Succeeded.", /*req, string, error message*/
	"data":{
	"commandInterval": 5, /*req, integer, command request interval, range:[1,60], unit: second, desc: 5 seconds by default*/

	"errorDelay": 30, /*req, integer, reconnection interval, range:[30,300], unit: second, desc: 30 seconds by default; The time interval after which the device tries to reconnect to the platform after a network failure.*/ } }
Remarks for Response Success Message	After successful login, the platform responds with a 64-byte random string (*My-Custom-Challenge*) in the HTTP header, with valid characters ranging from 0-9 and a-f: HTTP/1.1 200 OK ... My-Custom-Challenge: 20678a373b5bc316149a7027422d1e8820678a373b5bc316149a7027422d1e88 1. *My-Custom-Challenge* remains valid until the device performs a logout operation or before the next operation. 2. Note that the *My-Custom-Auth* for the next operation depends on the previous *My-Custom-Challenge* value. Therefore, communication between the device and the platform must be sequential and does not support concurrent multiple communications. 3. To ensure security, the *My-Custom-Challenge* generated during platform-device interaction has a maximum validity period of 3 * *commandInterval* (command request interval) from the time it is generated. 4. If the current communication is interrupted due to network issues and the device does not receive the platform's next *My-Custom-Challenge* value, the device must re-initiate the registration authentication process.

Response Failure Message	{
	"status": 401, /*req, integer, status code*/
	"code": "0x0020000f", /*req, string, error code*/
	"errorMsg": "Invalid SessionID.", /*req, string, error message*/
	"data":{
	"lockStatus": "unlock", /*req, string, lock status, enum:[lock# locked, unlock# unlocked]*/
	"unlockTime ": 60, /*req, integer, lock duration, range:[10,3600], unit: second*/
	"retryLoginTime": 3, /*req, integer, remaining times of allowed login attempts, range:[0,5]*/
	}
	}

Remarks for Response Failure Message	Up to 5 attempts of incorrect *loginPassword* are allowed. After 5 failures, the device is locked. The lock duration is determined by the platform. The platform restricts attempts from the same IP and port; re-login is not allowed before the lock time expires.

After the device successfully logs in, it carries authentication information (*My-Custom-Auth*) in the HTTP request header for each command operation, event upload, or device logout request.

```
POST /iot/${DEVID}/global/0-global/model/service/operate/PUSH/CommandRequest HTTP/1.1
...
My-Custom-Auth: 472e0350c664568c5cd5bce2e215f1gh
```

Calculation method for *My-Custom-Auth*:

HexToString(SHA256(HexToString(SHA256(username + salt + password)) + My-Custom-Challenge))

Examples:

1. If *username* is admin, *password* is aksj13579, *My-Custom-Challenge* is 20678a373b5bc316149a7027422d1e8820678a373b5bc316149a7027422d1e88, and *salt* is e43edaeac631e5db1c331046294a3aee6787bea7d0a72b36ff405b83e4e6f9f3, then *My-Custom-Auth* is:

HexToString(SHA256(HexToString(SHA256("admin" +
"e43edaeac631e5db1c331046294a3aee6787bea7d0a72b36ff405b83e4e6f9f3" + "aksj13579")) +
"20678a373b5bc316149a7027422d1e8820678a373b5bc316149a7027422d1e88"))

The salt is the salt value returned by the platform when obtaining authentication information.

Each time the platform responds to a device's command operation, event upload, device logout request:

1. It carries a 64-byte random string (*My-Custom-Challenge*) in the HTTP response header.
2. To ensure security, it is recommended that the random string in each platform response be different.

```
HTTP/1.1 200 OK
...
My-Custom-Challenge: 20678a373b5bc316149a7027422d1e8820678a373b5bc316149a7027422d1e88
```

7.3 Command Operation

Refer to **F4: Get Command Request** in Chapter 6 System Architecture. Devices periodically send heartbeat requests to the platform at command request interval (*commandInterval*). The platform response has the following two scenarios:

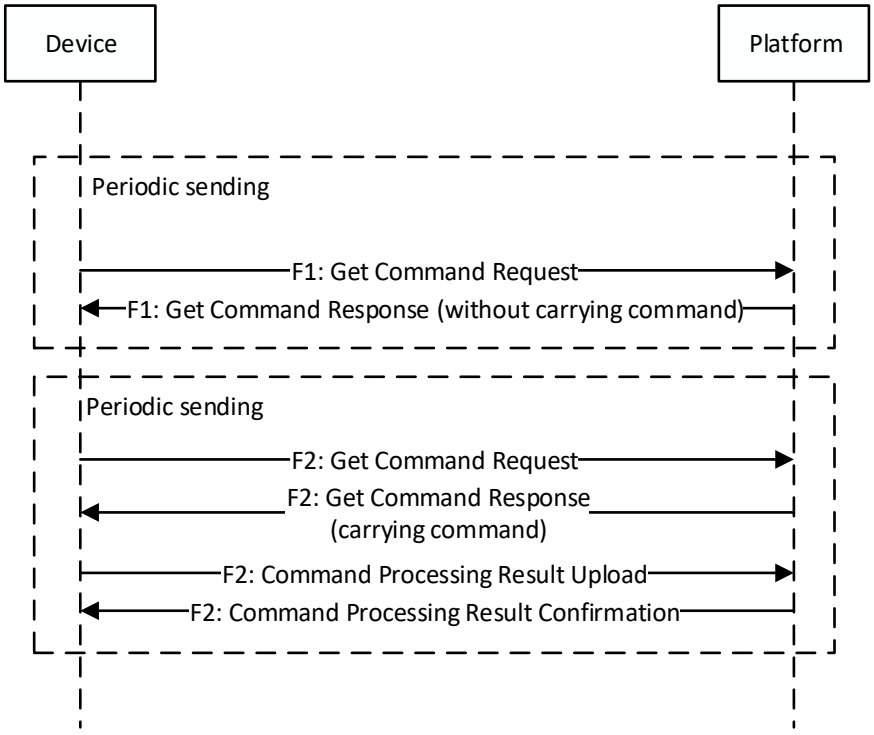


Figure 5 Flow of Get Command

- When the device sends a get command request, and the platform has no commands to carry:

API Function	Get Command
Request URL	POST /iot/${DEVID}/global/0-global/model/service/operate/PUSH/CommandRequest
Request Messages	None

Response Messages	{
	"status": 200, /*req, integer, status code*/
	"code": "0x00000000", /*req, string, error code*/
	"errorMsg": "Succeeded.", /*req, string, error message*/
	"commandNum": 0 /*req, integer, number of carried commands, range:[0,20]*/
	}

- When the device sends a get command request and the platform carries commands:

API Function	Heartbeat
Request URL	POST /iot/${DEVID}/global/0-global/model/service/operate/PUSH/CommandRequest
Request Messages	None

Response Messages	{
	"status": 200, /*req, integer, status code*/
	"code": "0x00000000", /*req, string, error code*/
	"errorMsg": "Succeeded.", /*req, string, error message*/
	"commandNum": 5, /*req, integer, number of carried commands, range:[0,20]*/
	"commandList":[{
	"UUID": "079f23cd-0988-459f-96f5-fa1c507dd07c", /*req, string, the unique identifier of the request command, range:[1,64]*/
	"URL": "PUT /ISAPI/System/deviceInfo", /*req, string, request command URL, range:[1,256]*/
	"dataFormat": "noData", /*req, string, data format, enum:[*noData*# no data, *boundaryData*#boundary format data, *xmlData*#XML format data, *jsonData*#JSON format data]*/
	"data": "" /*req, string, data content, desc: Base64 encoding is required*/
	}]
	}

API Function	Command Processing Result
Request URL	POST /iot/${DEVID}/global/0-global/model/service/operate/PUSH/CommandResult

Request Messages	{
	"commandNum": 5, /*req, integer, number of carried commands, range:[0,20]*/
	"commandList":[{
	"UUID": "079f23cd-0988-459f-96f5-fa1c507dd07c", /*req, string, the unique identifier of the request command, range:[1,64]*/

	"dataFormat": "noData", /*req, string, data format, enum:[boundaryData#boundary format data, xmlData#XML format data, jsonData#JSON format data]*/ "data": "" /*req, string, data content, desc: Base64 encoding is required*/ }] }

Response Messages	{
	"status": 200, /*req, integer, status code*/
	"code": "0x00000000", /*req, string, error code*/
	"errorMsg": "Succeeded.", /*req, string, error message*/
	"isPendingCommand": true /*req, boolean, Whether there are pending commands, desc: If returns true, it means the platform still has pending commands, the device should immediately send a Get Command request (not limited by *commandInterval* at this time)*/
	}

7.4 Event Upload

Refer to **F5: Event Upload** in Chapter 6 System Architecture. After a device triggers an event, it uploads the event in real-time. If the network is disconnected, all pending events are uploaded again once the network are reconnected.

API Function	Event Upload
Request URL	POST /iot/${DEVID}/global/0-global/model/service/operate/PUSH/Event

Request Messages	{
	"eventNum": 5, /*req, integer, number of carried events, range:[0,20]*/
	"eventList":[{
	"UUID": "079f23cd-0988-459f-96f5-fa1c507dd07c", /*req, string, the unique identifier of the event upload, range:[1,64]*/
	"dataFormat": "noData", /*req, string, data format, enum:[boundaryData#boundary format data, xmlData#XML format data, jsonData#JSON format data]*/
	"data": "" /*req, string, data content, desc: Base64 encoding is required (if dataFormat is xmlData or jsonData, data is in pure XML/JSON format; if dataFormat is boundaryData, data is subject to content descriptions in MIME_boundary in 4.3 Data Format)*/
	}]
	}

Response Messages	[{ "UUID": "079f23cd-0988-459f-96f5-fa1c507dd07c", /*req, string, the unique identifier of the event upload, range:[1,64]*/ "status": 200, /*req, integer, status code*/ "code": "0x00000000", /*req, string, error code*/ "errorMsg": "Succeeded." /*req, string, error message*/ }]

7.5 Device Logout

Refer to **F6: Logout Request** in Chapter 6 System Architecture. When the PUSH platform address/port changes or is disabled, perform logout first.

API Function	Log Out
Request URL	POST /iot/${DEVID}/global/0-global/model/service/operate/PUSH/Logout
Request Messages	None

Response Messages	{
	"status": 200, /*req, integer, status code*/
	"code": "0x00000000", /*req, string, error code*/
	"errorMsg": "Succeeded." /*req, string, error message*/
	}

7.6 Real-Time Live View

Real-time live view is implemented using the WebSocket streaming method.

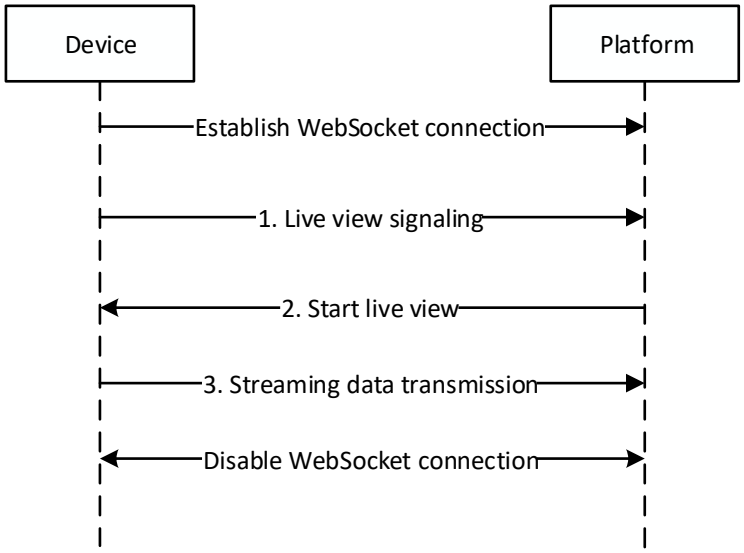


Figure 6 WebSocket Real-Time Live View

7.6.1 Live View Request

1. Before starting WebSocket real-time live view, the *StartPreview* command for the streaming service should be transmitted via the 7.3 Command Operation:
   (1) Request URL: POST /iot/${DEVID}/global/0-global/model/service/operate/LiveStreaming/StartPreview/<SessionID>;
   (2) The request message is empty (*dataFormat* is set to "noData" in the 7.3 Command Operation);
   (3) Response message example:

```
{
    "status": 200,      // Status code
    "code": "0x00000000",      // Error code
    "errorMsg": "Succeeded."      // Error description
}
```

   Note: *SessionID* identifies a single live view operation and is tightly bound to the subsequent live view WebSocket link. Length: 32 bytes (composed of 64 hexadecimal characters, digits 0-9, letters a-z, case-insensitive).
2. Before stopping WebSocket real-time live view, the *StopPreview* command for the streaming service should be transmitted via the 7.3 Command Operation:
   (1) Request URL: POST /iot/${DEVID}/global/0-global/model/service/operate/LiveStreaming/StopPreview/<SessionID>;
   (2) The request message is empty (*dataFormat* is set to "noData" in the 7.3 Command Operation);
   (3) Response message example:

```
{
```

```
    "status": 200,      // Status code
    "code": "0x00000000",      // Error code
    "errorMsg": "Succeeded."      // Error code description
}
```

Note: After the platform sends *StopPreview* to stop streaming, the platform does not actively disconnect the WebSocket link; the device must disconnect it. The platform can only consider the streaming normally closed after receiving the response to *StopPreview*.

The current error codes returned by *StartPreview* and *StopPreview* are as follows:

No.	status	code	errorMsg
1	200	0x00000000	Succeeded
2	400	0x00600001	The number of streaming channels exceeded limit.
3	400	0x00100003	This function is not supported.

7.6.2 Establishing Connection

1. Example of real-time live view signaling:

- Request message

```
{
    "sequence": 1, // Required, sequence number. The initial value is a random number, increased for each subsequent request (max value 65535, restarts from 0 after reaching the maximum).
    "cmd": "realplay", // Required, request type: "realplay" (real-time live view).
    "DEVID": "", // Required, device serial number (can be letters, digits, or a combination of both).
    "My-Custom-Auth": "472e0350c664568c5cd5bce2e215f1gh", // Optional, authentication information (used in the same way as that for HTTP).
    "sessionID": "0040a88c2ade10f8214ce2e05d14541bbd76523f4129996cd957426f0d376294" // Required, session ID, must match the StartPreview API.
}
```

Note: The *"sequence"* field is used to distinguish the order of multiple requests over the same client connection. It is required, and the device assigns an independent number for each WebSocket connection.

- Response message

```
{
    "sequence": 1, // Required, sequence number. The initial value is a random number, increased for each subsequent request (max value 65535, restarts from 0 after reaching the maximum).
    "status": 200,    // Required, status code.
    "code": "0x00000000",    // Required, error code.
    "errorMsg":"Succeeded.", // Required, error description.
    "My-Custom-Auth": "472e0350c664568c5cd5bce2e215f1gh", // Optional, authentication information (used in the same way as that for HTTP).
    "sessionID": "0040a88c2ade10f8214ce2e05d14541bbd76523f4129996cd957426f0d376294" // Required, session ID, must match the StartPreview API.
}
```

2. **Example of starting live view**:

- Request message

```
{
    "sequence": 2, // Required, sequence number. The initial value is a random number, increased for each subsequent request (max value 65535, restarts from 0 after reaching the maximum).
    "url": "live://<ip>:<port>/<channel>/<streamType>", // Required, stream URL. For streaming via direct-connection, the URL format is as follows (streaming media URLs are obtained from the platform, and passed through as obtained. The ,protocol does not define it). "channel" is the channel number, and "streamType" is the stream type (1-main stream, 2-sub-stream).
    "protocolType": "OTAP", // Optional, protocol type. "OTAP" (error codes in the response must comply with OTAP protocol requirements; if this field is not provided, error codes in the response must comply with ISAPI protocol requirements).
    "sessionID": "0040a88c2ade10f8214ce2e05d14541bbd76523f4129996cd957426f0d376294" // Required, session ID, must match the StartPreview API.
}
```

Notes:

1. The "sequence" field is used to distinguish the order of multiple requests in a unified client connection. It is required, and the device assigns an independent number for each WebSocket connection.
2. For "*url*" field: "ip" is the device IP address; "*port*" is the device port (the device does not need to parse "ip" and "port" for now); "*channel*" defaults to 1; and "*streamType*" defaults to 1.

- Response message

```
{
    "sequence": 2, // Required, sequence number. The initial value is a random number, increased for each subsequent request (max value 65535, restarts from 0 after reaching the maximum).
    "status": 200,    // Required, status code.
    "code": "0x00000000",    // Required, error code.
    "errorMsg":"Succeeded.", // Required, error description.
    "sdp":"" // Required, SDP data. See the SDP data format section for details.
}
```

SDP data format:

```
m=video 0 RTP/AVP 96
a=rtpmap:96 H264/90000
a=control:trackID=1
a=fmtp:96 profile-level-id=4D0014;packetization-mode=0;sprop-parameter-sets=Z0LAH4iLUCgC3QgAADhAAAr8gBA=,aM44gA=
```

Notes:

1. The SDP is limited to describing a single media stream, with no need for negotiation between parties.
2. The stream format for real-time live view is H264.
3. The SDP does not support external connection push streams (i.e., external TCP/UDP communication for push streaming is not allowed). Only push streaming within the current WebSocket connection is permitted.
4. Frame format in WebSocket: JSON signaling messages use text frames, while stream data uses binary frames.

7.6.3 Sending Streaming Data

Example of message for requesting streaming data:

```
{
    "sequence": 3, // Required, sequence number. The initial value is a random number, increased for each subsequent request (max value 65535, restarts from 0 after reaching the maximum).
    "cmd": "play" // Required, request type: "play" (request for stream data).
    "sessionID": "0040a88c2ade10f8214ce2e05d14541bbd76523f4129996cd957426f0d376294" // Required, session ID, must match the StartPreview API.
}
```

Stream format is as follows: (Simply keep sending continuously)

```
Streaming data
...
Streaming data
```

In practice, stream data is sent over WebSocket frames. Theoretically, a single WebSocket frame can transmit over 500 MB of data.

Currently, for RTP streams, each RTP packet must be sent as a separate WebSocket frame; for PS streams, this requirement is not mandatory.

7.7 Two-way Audio

Two-way audio is implemented using WebSocket streaming.

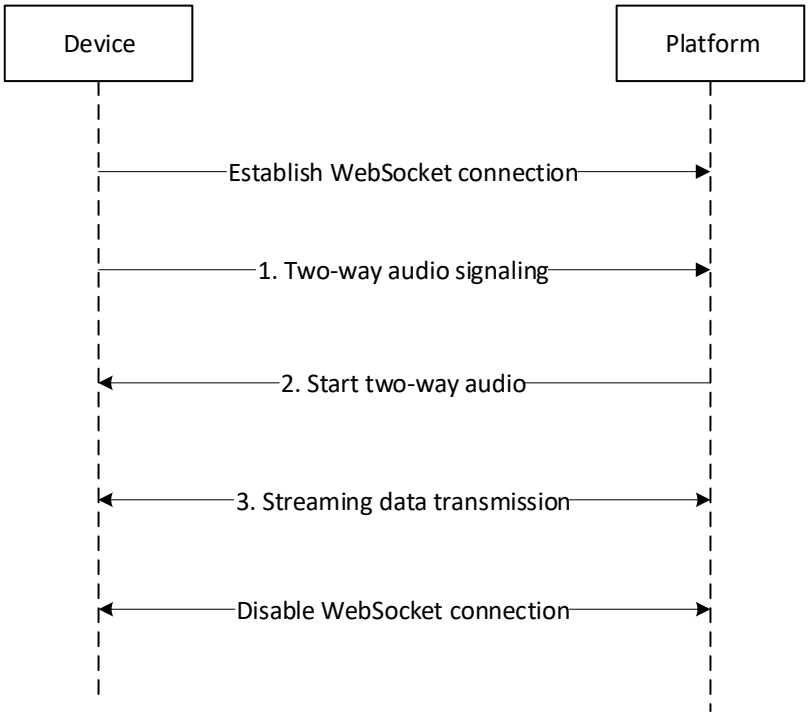


Figure 7 WebSocket Two-way Audio

7.7.1 Two-way Audio Request

1. Before starting WebSocket two-way audio, the *StartVoiceTalk* command should be sent to the streaming media via the 7.3 Command Operation:
   (1) Request URL: POST /iot/${DEVID}/global/0-global/model/service/operate/VoiceTalk/StartVoiceTalk/<SessionID>;
   (2) Request body is empty (set "*dataFormat*" to "noData" in the 7.3 Command Operation);
   (3) Response message example:

```
{
    "status": 200,      // Status code
    "code": "0x00000000",      // Error code
    "errorMsg": "Succeeded."      // Error description
}
```

   Note: *SessionID* identifies a single two-way audio operation and is tightly bound to the subsequent two-way audio WebSocket connection. Length: 32 bytes (composed of 64 hexadecimal characters, digits 0-9, letters a-z, case-insensitive).
2. Before stopping WebSocket two-way audio, the *StopVoiceTalk* command for the streaming service should be transmitted via the 7.3 Command Operation:
   (1) Request URL: POST /iot/${DEVID}/global/0-global/model/service/operate/VoiceTalk/StopVoiceTalk/<SessionID>;
   (2) The request message is empty (set "*dataFormat*" is set to "noData" in the 7.3 Command Operation);
   (3) Response message example:

```
{
    "status": 200,      // Status code
    "code": "0x00000000",      // Error code
    "errorMsg": "Succeeded."      // Error code description
}
```

Note: After the platform sends *StopVoiceTalk* to stop streaming, the platform will not actively close the WebSocket connection; the device should disconnect. The platform can only consider the stream normally closed upon receiving the response to *StopVoiceTalk*.

Error codes returned by *StartVoiceTalk* and *StopVoiceTalk* are as follows:

No.	status	code	errorMsg
1	200	0x00000000	Succeeded
2	400	0x00600001	The number of streaming channels exceeded limit.
3	400	0x00100003	This function is not supported.

7.7.2 Establishing Connection

1. **Example of two-way audio signaling:**

- Request message

```
{
    "sequence": 1, // Required, sequence number. The initial value is a random number, increased for each subsequent request (max value 65535, restarts from 0 after reaching the maximum).
    "cmd": "voicetalk", // Required, request type: "voicetalk" (two-way audio).
    "DEVID": "", // Required, device serial number (can be letters, digits, or a combination of both).
    "My-Custom-Auth": "472e0350c664568c5cd5bce2e215f1gh", // Optional, authentication information (used in the same way as that for HTTP).
    "sessionID": "0040a88c2ade10f8214ce2e05d14541bbd76523f4129996cd957426f0d376294" // Required, session ID, must match the StartVoiceTalk API.
}
```

Note: The "sequence" field is used to distinguish the order of multiple requests over the same client connection. It is required, and the device assigns an independent number for each WebSocket connection.

- Response message

```
{
    "sequence": 1, // Required, sequence number. The initial value is a random number, increased for each subsequent request (max value 65535, restarts from 0 after reaching the maximum).
    "status": 200,    // Required, status code.
    "code": "0x00000000",    // Required, error code.
    "errorMsg":"Succeeded.", // Required, error description.
    "My-Custom-Auth": "472e0350c664568c5cd5bce2e215f1gh", // Optional, authentication information (used in the same way as that for HTTP).
    "sessionID": "0040a88c2ade10f8214ce2e05d14541bbd76523f4129996cd957426f0d376294" // Required, session ID, must match the StartVoiceTalk API.
}
```

2. **Example of starting two-way audio**:

- Request message

```
{
    "sequence": 2, // Required, sequence number. The initial value is a random number, increased for each subsequent request (max value 65535, restarts from 0 after reaching the maximum).
    "url": " live://<ip>:<port>/<voicetalkchannel> ", // Required, stream URL. For streaming via direct-connection, the URL format is as follows (streaming media URLs are obtained from the platform, and passed through as obtained format. The protocol does not define it). " voicetalkchannel" is the two-way audio channel number.
    "sessionID": "0040a88c2ade10f8214ce2e05d14541bbd76523f4129996cd957426f0d376294" // Required, session ID, must match the StartVoiceTalk API.
}
```

Notes:

1. The "*sequence*" field is used to distinguish the order of multiple requests in a unified client connection. It is required, and the device assigns an independent number for each WebSocket connection.
2. For "*url*" field: "*ip*" is the device IP address; "*port*" is the device port (the device does not need to parse "ip" and "port" for now), and " *voicetalkchannel*" defaults to 1.

- Response message

```
{
    "sequence": 2, // Required, sequence number. The initial value is a random number, increased for each subsequent request (max value 65535, restarts from 0 after reaching the maximum).
    "status": 200,    // Required, status code.
    "code": "0x00000000",    // Required, error code.
    "errorMsg":"Succeeded.", // Required, error description.
    "sdp":"" // Required, SDP data. See the SDP data format section for details.
}
```

SDP data format:

```
v=<protocol version>        // Current protocol version is 0.
o=<username> <session id> <session version> <network type> <address type> <unicast address> // Fields separated by spaces. <username> is the user name on the source host, if no user name, use "-", spaces are not allowed. <session id> is a numeric string, represented as an NTP format timestamp. <session version> is the current session version number, represented as an NTP format timestamp, it increments when session data changed. <network type> is a text string, "IN" for internet. <address type> is a text string: "IP4", "IP6". <unicast address> is the host address creating the session.
s=<session name>        // Session name is a text string, each session has exact one "s=" field, value cannot be empty, use a space if no session name.
c=<network type> <address type> <connection address>      // <connection address> is the media data source address.
a=<attribute>        // "sendrecv" indicates bidirectional send/receive.
m=<media type> <port> <protocol> <format type>        // "m" is media description; <media type> include video, audio, text, application, message; audio is "audio"; <port> is the port for sending media stream; <protocol> include UDP, RTP/AVP, RTP/SAVP, here it's RTP/AVP; <format type>: when protocol is RTP/AVP or RTP/SAVP, it represents the RTP payload format number.
c=<network type> <address type> <connection address>        // <connection address> is the media data source address.
a=<attribute>        // "a=rtpmap:" attribute dynamically maps media format number to media format;
```

```
RTP payload number and encoding format: a=rtpmap:<payload type> <encoding name>/<clock rate>[/<encoding parameters>]
a=<attribute>       // "control:trackID=1" indicates the stream track number; two-way audio has one audio track.
a=*<attribute>      // "*" indicates the parameter is optional; "a=fmtp:<format> <format specific parameters> encoding and decoding parameters"; e.g., a=fmtp:98 bitrate=48000
```

SDP Data Example:

```
v=0
o=- 10002208 10002208 IN IP4 0.0.0.0
s=sessionName
c=IN IP4 0.0.0.0
a=sendrecv
m=audio 0 RTP/AVP 0
c=IN IP4 0.0.0.0
a=rtpmap:98 G7221/16000
a=control:trackID=1
```

Notes:

1. Two-way audio involves bidirectional interactive audio stream data. “a=sendrecv” in the SDP indicates both ends send and receive data simultaneously.
2. The server sends audio type and encoding information via SDP (e.g., encoding type G.722.1, AAC, PCM, G.711U, etc., sample rate 16kHz, 32kHz, etc.; now only supports G.711U). The client must use consistent settings. Both parties send and receive audio data according to the agreed audio type.
3. The SDP is limited to describing a single media stream; no negotiation is required.
4. The SDP does not support external connection push streams (i.e., external TCP/UDP communication for push streaming is not allowed). Only push streaming within the current WebSocket connection is permitted.
5. Frame format in WebSocket: JSON signaling messages use text frames, while stream data uses binary frames.

7.7.3 Sending Streaming Data

Since WebSocket is a full-duplex connection, both the client and server can send audio stream data and receive data from the peer on a single two-way audio connection.

After the server sends the SDP information to the client, the server can start sending audio data. Similarly, after the client receives the SDP information, it can send audio data to the peer.

Stream format is as follows: (Simply keep sending continuously)

```
Streaming data
...
Streaming data
```

In practice, stream data is sent over WebSocket frames. Theoretically, a single WebSocket frame can transmit over 500 MB of data.

Currently, for RTP streams, each RTP packet must be sent as a separate WebSocket frame; for PS streams, this requirement is not mandatory.

7.8 Video Intercom

Devices initiate the live view with the platform via local UI/key calling.

- The platform answers the call, enabling two-way audio between the platform and the device.
- The platform reject a call, the live view between the platform and the device is terminated without two-way audio.

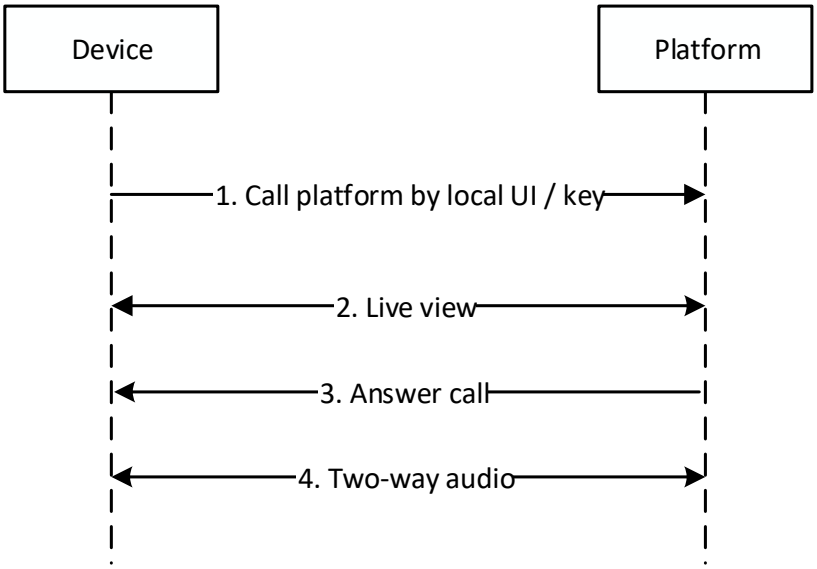


Figure 8 Video Intercom

1. Call platform by Local UI/Key: Triggers an event upload. The event type is a video intercom interaction event (*voiceTalkEvent*). Refer to 7.4 Event Upload.
2. Real-time live view: WebSocket real-time live view is conducted between the device and the platform. Refer to 7.6 Real-Time Live View (perform steps 7.6.2 and 7.6.3; the 7.6.1 Live View Request is not required).
3. Answer Call: After the platform receives the intercom interaction event, it triggers a command operation. The operation responding to the event is to configure the call signaling interaction (PUT /ISAPI/VideoIntercom/callSignal?format=json). Refer to 7.3 Command Operation.
4. Two-way Audio: WebSocket two-way audio is conducted between the device and the platform. Refer to 7.7 Two-way Audio (execute steps 7.7.2 and 7.7.3; the 7.7.1 Two-way Audio Request is not required. Note: The *SessionID* is generated by the device).

7.9 Local Upgrade

The main process when a device performs a local upgrade is as follows.

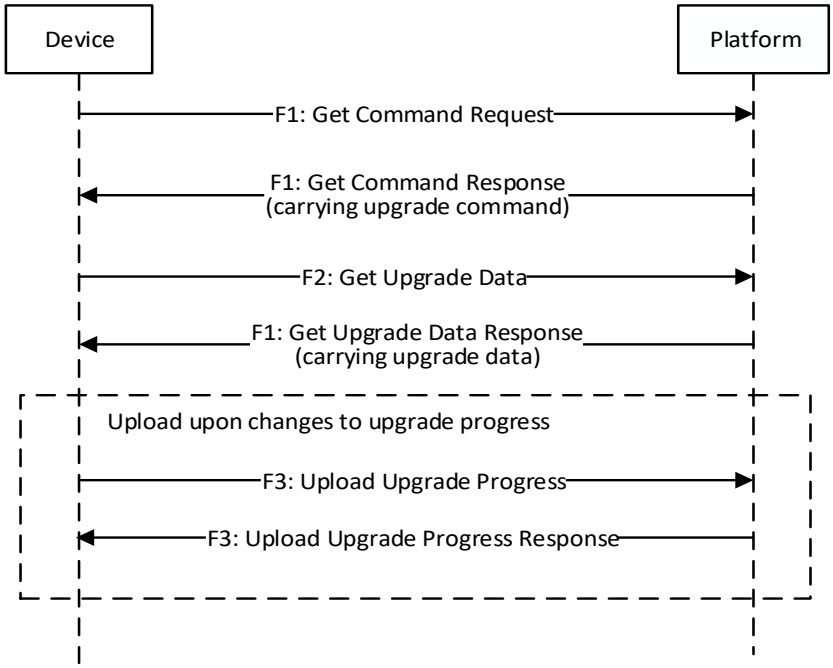


Figure 9 Local Upgrade

1. When the device sends a command getting request and the platform carries an upgrade command (only one upgrade command can be carried, and no other commands can be carried simultaneously):

API Function	Heartbeat
Request URL	POST /iot/${DEVID}/global/0-global/model/service/operate/PUSH/CommandRequest
Request Messages	None
Response Messages	{ "status": 200, /*req, integer, status code*/ "code": "0x00000000", /*req, string, error code*/ "errorMsg": "Succeeded.", /*req, string, error message*/ "commandNum": 1, /*req, integer, number of carried commands (the value can only be "1" if the upgrade command is carried)*/ "commandList":[{ "UUID": "079f23cd-0988-459f-96f5-fa1c507dd07c", /*req, string, the unique identifier of the request command, range:[1,64]*/ "URL": "PUT /ISAPI/System/updateFirmware", /*req, string, URL of the request command (upgrade command URL), range:[1,256]*/ "dataFormat": "noData", /*req, string, data format, enum:[noData# no data]*/ "data": "" /*req, string, data content (the value is empty when the upgrade command is carried)*/ }] }

2. Get upgrade data (The device sends this request immediately after detecting the upgrade command request):

API Function	Get Upgrade Data
Request URL	POST /iot/${DEVID}/global/0-global/model/service/operate/PUSH/UpgradeData

Request Messages	{
	"readyUpgrade": true, /*req, boolean, whether the device is currently ready for upgrade*/
	"reason": "" /*opt, string, the reason if the device cannot upgrade (returned when *readyUpgrade* is false)*/
	}

Response Message 1 (Returned when *readyUpgrade* is true)	Binary upgrade data

Response Message 2 (Returned when *readyUpgrade* is false)	{
	"status": 200, /*req, integer, status code*/
	"code": "0x00000000", /*req, string, error code*/
	"errorMsg": "Succeeded." /*req, string, error message*/
	}

3. Upload upgrade progress (The device sends this request immediately after detecting a change in the upgrade progress):

API Function	Upload Upgrade Progress
Request URL	POST /iot/${DEVID}/global/0-global/model/service/operate/PUSH/UpgradeStatus

Request Messages	{
	"status": 200, /*req, integer, status code*/
	"code": "0x00000000", /*req, string, error code*/
	"errorMsg": "Succeeded.", /*req, string, error message*/
	"upgrading": true, /*opt, boolean, whether the device is being upgraded (returned when status value is 200)*/
	"percent": 0, /*opt, integer, device upgrade progress (returned when status is 200), range:[0,100]*/
	"reboot": true /*opt, boolean, whether the device will reboot automatically after upgrade is complete (returned only when status is 200 and percent reaches 100)*/
	}

Response Messages	{
	"status": 200, /*req, integer, status code*/
	"code": "0x00000000", /*req, string, error code*/
	"errorMsg": "Succeeded." /*req, string, error message*/
	}

